

Git, GitHub, worktrees and review

Module 5 · from a local folder to a reviewed pull request

Lab: L03 Git, GitHub, worktrees and review

OBJECTIVES

What you will be able to do

- Put a project under Git with a safe .gitignore, using Claude
- Create a GitHub repository and push to it with the gh CLI
- Open a pull request from a feature branch
- Build two features in parallel with worktrees
- Review changes with /code-review and /security-review
- Post review findings on a pull request, fix them, and merge

5.1 · THE FLOW

From a folder to a reviewed pull request

1 **Local folder**

git init, .gitignore, first commit

→

2 **GitHub**

gh repo create, push main

→

3 **Branches and worktrees**

one feature per worktree

4 **Pull request**

push the branch, gh pr create

→

5 **Review**

/code-review, /security-review

→

6 **Merge**

after a person approves

Claude runs every git and gh command for you, as you. You approve each push and read each diff (M02, M04).

5.1 · BEFORE YOU START

What you need

Git

git --version. On Windows, Git for Windows (M02)

A GitHub account

Free is enough. Your organisation may give you one

The gh CLI, signed in

Install from cli.github.com, then gh auth login. Check: gh auth status

CHECK IN ONE GO

```
git --version
gh --version
gh auth status
```

Claude uses gh for GitHub work: creating repos, pull requests, reading PR comments, posting reviews.

Put the project under Git

PROMPT

```
Initialise git in this folder on branch main.
Create a .gitignore for a Python (uv) backend
and a React (Vite) frontend. Ignore:
.venv, __pycache__, node_modules, dist,
.env*, backend/todo.db, CLAUDE.local.md,
.claude/settings.local.json, .claude/worktrees/
Then show me git status. Do not commit yet.
```

Never in the repo

- .env files, API keys
- Virtual envs, node_modules
- The local backend/todo.db
- Your CLAUDE.local.md
- Your settings.local.json
- Worktree folders

THEN

```
Review git status yourself, then: Make the first commit: "Initial commit: backend and frontend".
```

5.3 · GITHUB

Create the GitHub repository and push

PROMPT

```
Create a private GitHub repository named todo-app with gh, add it as origin,
and push main. Then show me the remote URL with git remote -v.
```

WHAT CLAUDE RUNS (YOU APPROVE IT)

```
gh repo create todo-app --private --source=. --remote=origin --push
```

Private first

Make it public later, on purpose

It runs as you

gh uses your GitHub login

No force push

Deny git push --force (M8)

Commits Claude writes, you own

HABIT	WHY
Small commits, one step each	Easy to read, revert and bisect
Read git diff before every commit	You are accountable for what you commit (M04)
Claude writes the message, you check it	One line saying what and why
AI-assisted attribution stays on	Claude adds a Co-Authored-By line by default (M18)

PROMPT

```
Show me git diff --staged. Then commit it with a one-line message in the
imperative mood, under 60 characters, that says what changed and why.
```

Feature branch to pull request

1	`git switch -c show-open-count`	Branch first
2	**Plan, implement, verify**	The M04 rhythm, with commits
3	`git push -u origin show-open-count`	Push the branch
4	`gh pr create`	Open the pull request

PROMPT

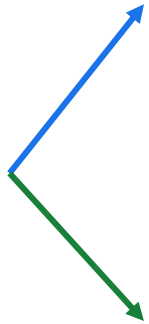
```
Push this branch and open a pull request with gh against main.
Title: one line. Body: what changed, why, how to test (uv run pytest -q in backend/),
and anything you were unsure about. Do not merge it.
```

Worktrees: one checkout per feature

A worktree is a second folder with its own files and branch, sharing the same repository history. Two Claude sessions, two features, no collisions.

todo-app/

main checkout
branch: main
.git (shared history)



.claude/worktrees/priority/

`branch: worktree-priority`

Session 1: priority field, sort, tests (backend/ only)

.claude/worktrees/search/

`branch: worktree-search`

Session 2: SearchBox.jsx, client-side filter (frontend/ only)

Edits in one worktree never touch the other. Each ends as its own pull request.

Start two sessions in parallel

TERMINAL 1

```
claude --worktree priority
```

TERMINAL 2

```
claude -w search
```

DETAIL	WHAT HAPPENS
Folder and branch	.claude/worktrees/<name>/ on a new branch worktree-<name>
Starting point	The repository's default branch on GitHub (fresh), not your current work
From a pull request	`claude --worktree "#42"`
No name given	Claude picks one, such as bright-running-fox

Run claude once in the repo first to accept the trust prompt. Add .claude/worktrees/ to .gitignore.

Each worktree is a fresh checkout

Not copied

Ignored files: .env, .venv, node_modules. Install dependencies in each worktree

.worktreeinclude

Lists ignored files to copy into every new worktree, in .gitignore syntax

.WORKTREEINCLUDE

```
.env
backend/.env.local
```

FIRST PROMPT IN A NEW WORKTREE

```
Set up this worktree: run uv sync in backend/
and npm install in frontend/, then run uv run pytest -q
in backend/ and tell me the result.
```

Same repo, same plugins and saved approvals; only files and branch are separate.

Finish, keep or remove a worktree

1	**Commit and push**	`git push -u origin worktree-priority`
2	**Open the pull request**	`gh pr create, from inside the worktree`
3	**Exit the session**	Clean: removed automatically. Work left: Claude asks keep or remove
4	**Come back later**	`claude --worktree priority --resume`

BY HAND, WITH GIT

```
git worktree list
git worktree remove .claude/worktrees/priority
```

Other ways to get a worktree

WAY	USE IT WHEN
Ask in plain words	"work on this in a worktree" during a session; Claude creates and enters one
Subagent with isolation: worktree	A subagent that edits many files in its own worktree (M7)
/batch <instruction>	One big change split into 5 to 30 units, each in its own worktree
Background sessions	claude --bg starts in a worktree

Rules for parallel features

Separate areas

Pick features that touch different folders: backend/ vs frontend/

Merge one at a time

Merge the first PR, then rebase the second on main and re-run the tests

Usage adds up

Each session draws on your plan separately

Review each PR alone

One diff, one review, one decision

```
Prompt: Rebase this branch on origin/main, resolve conflicts, run the tests, and summarise what changed.
```

What AI-written code gets wrong

Missing tests

New endpoint or rule, no new test

Weakened tests

Assertion removed or loosened, skip

Hallucinated APIs

A function or option that does not exist

Over-broad changes

Unasked files; a tidy-up that changes a rule

Secrets and personal data

Keys in code, personal data in logs

Dependency additions

A new line in pyproject.toml or package.json

Silent behaviour change

> becomes >=, an error is swallowed

Skipped invariants

Missing validation; local time instead of UTC



The one habit: read the diff of the tests before the diff of the code.

Ways to get a second look

TOOL	WHO STARTS IT	WHAT IT DOES
`/code-review`	You, or Claude when asked	Correctness bugs in your diff, a branch, a path or a PR. Runs as a background subagent. Any plan
`/security-review`	You	Security issues in your branch vs origin's default branch: injection, auth, data exposure
`/code-review ultra`	Only you	Deep multi-agent review in the cloud. May bill usage credits
Your own reviewer	You or Claude	A read-only subagent or skill with your rules

/review is an alias of /code-review.

/code-review: levels and targets

```
/code-review [low|medium|high|xhigh|max|ultra] [--fix] [--comment] [--max-findings n|all]
[pr#|branch|path]
```

TYPE	TO REVIEW
`/code-review`	Branch + uncommitted changes
`/code-review 12`	Pull request 12
`/code-review main...feat`	One branch against another
`/code-review backend/app`	A folder
`/code-review HEAD~5..HEAD`	The last 5 commits

Levels and flags

- low: fewer, surer findings
- medium to max: wider coverage
- No level: reuses the last one you typed
- --fix: apply the findings. /rewind does not undo them; use git
- --comment: post findings on the PR

5.7 · SECURITY REVIEW

/security-review

Reviews the diff between your branch and origin's default branch for security risks. It needs an origin remote, one more reason to push early.

EACH FINDING CARRIES

Severity

How serious it is

Confidence

How sure the review is

Category

e.g. sensitive_data_exposure

Exploit scenario

How it could be abused

EXAMPLE FINDING (ILLUSTRATIVE)

```
[High · confidence 0.9 · sensitive_data_exposure] backend/app/main.py:42
The to-do title is written to the log. Titles can hold personal data (R8).
```



Run it on any branch that touches input handling, login, logging or data access. It overlaps with /code-review; neither replaces the other.

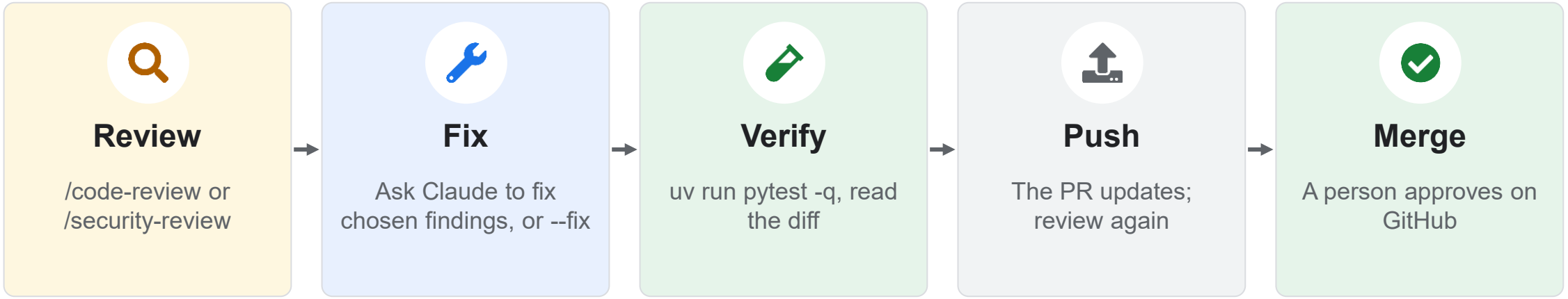
Review a pull request, with inline comments

```
/code-review high 12 --comment
```

1. **Claude fetches PR 12 with gh** — Your gh login must reach the repository
2. **Reviews it in its own context** — A background subagent, separate from your work
3. **Posts each finding on its line** — As you, on GitHub

⚠ **The comments carry your name. For a teammate's PR, run it once without --comment, read the findings, then post.**

Fix, re-review, merge



PROMPT

```
Fix findings 1 and 3 from the review only. Do not touch other files. Run uv run pytest -q,
then commit "Address review findings" and push.
```

Claude comments; a person approves. Branch protection on main makes that a rule, not a habit.

Teach the reviewer your rules

/code-review follows your CLAUDE.md. It does not know your team's rules unless they are written there.

CLAUDE.MD: A REVIEW SECTION

```
## Review rules
- Dates are stored in UTC, ISO 8601 (R1).
- No clock inside rules.py: pass today in (R7).
- Never log to-do titles; log the id (R8).
- Titles 1-200 chars; archive, never delete.
- No new dependency without a person's approval.
- Every new endpoint has a test in backend/tests/.
```

Note

REVIEW.md configures only the managed Code Review service on GitHub. The local /code-review reads CLAUDE.md.

A rule a reviewer can check (M3) is a rule /code-review can enforce in comments.

REFERENCE

Prompts: Git and GitHub

SET UP GIT

```
Initialise git on main. Create a .gitignore for a Python (uv) backend and a React (Vite) frontend, including .env*, .venv, node_modules, dist, CLAUDE.local.md, .claude/settings.local.json and .claude/worktrees/. Show git status; do not commit yet.
```

PUSH TO GITHUB

```
Create a private GitHub repo named todo-app with gh, set it as origin, push main, and show git remote -v.
```

COMMIT

```
Show git diff --staged, then commit with a one-line imperative message under 60 characters.
```

OPEN A PR

```
Push this branch and open a PR against main with gh. Body: what, why, how to test, open questions. Do not merge.
```

REFERENCE

Prompts: worktrees and parallel work

START (SHELL)

```
One per terminal:

claude --worktree priority
claude --worktree search
```

SET UP

```
Set up this worktree: uv sync in backend/, npm
install in frontend/, then run uv run pytest -q
and report.
```

IN-SESSION

```
Work on the search box in a new worktree named
search, then open a PR from it.
```

CATCH UP

```
Rebase this branch on origin/main, resolve
conflicts, run the tests, and summarise what
changed.
```

REFERENCE

Prompts: code and security review

REVIEW MY BRANCH

```
/code-review high
```

SECURITY PASS

```
/security-review
```

REVIEW A TEAMMATE'S PR

```
/code-review high 12          (read first)
/code-review high 12 --comment   (then post)
```

FIX CHOSEN FINDINGS

```
Fix findings 1 and 3 only. Do not edit any test
unless the finding is about that test. Run uv run
pytest -q and show me the diff.
```

REFERENCE

Commands

SHELL	
`gh auth login`	Sign gh in to GitHub
`gh repo create ... --push`	Create the repo and push
`gh pr create`	Open a pull request
`claude -w <name>`	Session in a new worktree
`claude -w "#12"`	Worktree from PR 12
`git worktree list`	List worktrees
`git worktree remove <path>`	Remove one

IN A SESSION	
`/diff`	Show what changed
`/code-review [level]`	Review the diff
`/code-review 12 --comment`	Review PR 12, post comments
`/code-review --fix`	Review and apply fixes
`/security-review`	Security review of the branch
`/batch <instruction>`	Split a big change across worktrees
`/install-github-app`	Claude GitHub app + Actions (M19)

CHECK YOUR UNDERSTANDING

Question 1

Which files must never be committed, and how do you make sure Claude does not commit them?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

Why does /security-review fail on a project that was never pushed to GitHub?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

You start two worktree sessions. Where do they live, and which branch does each start from?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

A new worktree's tests fail with "module not found" and a missing .env. Why, and what is the fix?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 5

You ran /code-review --fix and want to undo one change. Why does /rewind not help?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 6

You want Claude's findings on a teammate's pull request. What do you run first, and why not --comment straight away?

Think first, then we discuss.